license-administration

License Administration (Operator Guide)

This document is for the Preston-Check operator (you) who issues licenses to paying customers. It is not user-facing — keep it in the repository for internal reference but do not distribute the private signing key it references.

One-time setup

Generate the Ed25519 signing keypair on a secure machine, ideally an offline laptop or a dedicated workstation. Run:

```
tools/setup-signing-key.sh
```

This produces two files: `~/.preston-check/keys/private.pem` (NEVER commit, NEVER share, NEVER copy unencrypted to cloud storage) and `~/.preston-check/keys/public.pem`. The public key is also copied into `lib/license_pubkey.pem` in the repository so all customer instances can verify licenses offline.

Back up the private key immediately to a secure offline location: encrypted USB stored in a safe, hardware security module if you have one, or at minimum an encrypted backup with a strong passphrase recorded in your password manager. Losing the private key invalidates every license you ever issue, with no recovery path.

Commit the public key (`lib/license_pubkey.pem`) to the repository and push. This needs to happen before you can issue licenses to customers, because customer instances load the public key from this file.

Issuing a license

```
tools/issue-license.sh \
  --customer acme-fintech \
  --email ops@acme.example \
  --tier pro \
  --expires 2027-05-03 \
  --max-repos 5 \
  --output acme-fintech.license
```

The output file is a PEM-style envelope containing the base64-encoded JSON payload and the base64-encoded Ed25519 signature. Email it to the customer with installation instructions:

```
mkdir -p ~/.preston-check
cp acme-fintech.license ~/.preston-check/license

# Or set the path explicitly:
export PRESTON_LICENSE=/path/to/acme-fintech.license
```

For CI integration, customers can store the license content as a GitHub Actions secret (`PRESTON_CHECK_LICENSE`) and reference it in the workflow as `license: ${{ secrets.PRESTON_CHECK_LICENSE }}`.

License lifecycle

Licenses have a hard expiry date. Strict enforcement applies: an expired Pro/Enterprise license falls back to Free tier with a clear error message at startup; paid features stop working until renewal. The 30-day pre-expiry warning window prints a renewal banner in every report so customers see the upcoming expiration well before it hits.

To renew, issue a new license with the same `--customer` ID and a new expiry date. The customer replaces their old license file with the new one. There is no “renewal” step on the issuer side beyond issuing a fresh license — the new license simply overwrites the old.

To revoke a license before expiry, you currently have no built-in mechanism. Add one of three options when you need this: 1. Roll the signing key. Re-issue licenses for everyone except the revoked customer. This is heavyweight and rare. 2. Maintain a revocation list at a public URL the runner consults at startup (breaks airgap). Not recommended for this product positioning. 3. Reduce expiry to short windows (e.g., 30 days) so revocation happens naturally via non-renewal. Practical for high-risk relationships.

Tier semantics

When issuing licenses, the `--tier` flag must be exactly `pro` or `enterprise`. Free tier requires no license, so never issue a license with `tier: free`.

Pro customers get all checks regardless of `min_tier` declared in metadata, plus the Pro-only audit-package features (compliance evidence layer, multi-repo dashboard, branded reports). Pricing is $999/repo/yr or $4,999/yr unlimited.

Enterprise customers get everything Pro provides plus white-label report branding (`brand_name`, `brand_logo_url`, `brand_footer`, `brand_color` config keys), SSO, custom check authoring support, and dedicated success contact. Pricing starts at $29,999/yr.

Setting up customer-portal automation

The `tools/issue-license.sh` script is designed to be wrapped by an automated customer portal. The wrapper would handle:

1. Stripe / payment-provider checkout completion webhook.
2. Customer-record creation in your CRM (or a lightweight database).
3. Calling `tools/issue-license.sh` with the appropriate flags based on the purchased tier and term.
4. Emailing the resulting `.license` file to the customer.
5. Storing the license-issuance event for renewal tracking.

The simplest implementation is a small Node.js or Python service that exposes a `/webhook/stripe` endpoint, executes the issue-license shell script, and posts the output via SendGrid or AWS SES. Sub-100 lines of code; deployable to a single small VM or Cloudflare Worker plus shell-out.

For early-stage operations, you can issue licenses manually from your laptop. Manual issuance is fine for the first 50–100 customers; build automation when manual issuance becomes the bottleneck.

Privacy commitments to customers

When customers ask about privacy (and they will), the answer is:

The scanner runs entirely on their machine. It reads source files via `grep`, never sends source content anywhere, never opens network connections except for the explicitly opt-in telemetry that they control. License verification is fully offline; the public key is embedded in the repository, no phone-home, no

online activation servers. The `--airgap` flag guarantees zero outbound traffic for audit-sensitive environments.

You can verify all of this by reading the source code of the scanner. Apache 2.0 license, public repository, every line of code is auditable.

Operating discipline

Treat the signing key the way you would treat a code-signing certificate: it is the foundation of customer trust. If the key is compromised, attackers can issue fraudulent licenses claiming any tier they want. Detection is hard because each customer instance verifies locally with no central log.

Key-rotation plan: keep the current key in service for 18–24 months, then rotate. Issue all new licenses with the new key. Old licenses remain valid until expiry (because they verify against the previous public key, which can be retained as a secondary trust anchor by shipping both keys in `lib/`). After all old licenses expire, retire the previous key.

Compromise-response plan: revoke the existing key immediately by publishing a new public key and forcing all customers to receive fresh licenses. Communicate with affected customers within 24 hours. This is a serious incident; have a documented playbook before launch.

Compliance considerations for the issuer

You are running a commercial software vendor under Preston-Check. Expect customers to ask for:

- A SOC 2 Type II report covering your customer-portal infrastructure (the audit-package layer SaaS, not the open-source scanner).
- A DPA (Data Processing Agreement) under GDPR if you have EU customers and process any of their personal data (license records, contact info, telemetry if opted in).
- Vendor security questionnaires (SIG, CAIQ) for enterprise procurement.
- An MSA / EULA with indemnification, limitation of liability, SLA commitments. Use a fintech-savvy lawyer for the first draft.

These are normal commercial obligations and do not affect the open-source scanner. The scanner sits separately under Apache 2.0; the commercial agreements cover the audit-package SaaS and customer relationship.